

API Documentation

InfusioTech Restaurant Ordering SaaS | Version 1.0 | 2 October 2026

Purpose

Defines proposed API contracts for web and mobile clients. Paths are illustrative and should be versioned under `/api/v1`.

Conventions

- Base path: `/api/v1`. JSON request/response; ISO-8601 timestamps; money in integer minor units with ISO currency.
- Authenticated endpoints require bearer token and active tenant membership. Server resolves tenant scope from membership.
- Standard errors: 400 validation, 401 unauthenticated, 403 forbidden, 404 not found, 409 conflict, 422 business-rule failure, 429 rate limited, 500 unexpected.
- Use `request_id` for tracing. Mutating requests should accept Idempotency-Key where duplicate execution is possible.

Authentication & Membership

- POST `/auth/invitations/accept` — accept valid invitation and activate membership.
- GET `/me` — return identity, tenant memberships, role and effective permissions.
- POST `/tenants/{tenantId}/staff/invitations` — owner-authorized staff invite.
- GET `/tenants/{tenantId}/staff` — list staff; requires `staff.view`.
- PATCH `/tenants/{tenantId}/staff/{membershipId}` — update role/status/overrides; requires `staff.manage`.

Menu & Tables

- GET `/public/restaurants/{slug}/menu` — public active menu.
- POST `/tenants/{tenantId}/menu/items` — create item; requires `menu.create`.
- PATCH `/tenants/{tenantId}/menu/items/{itemId}` — update item; requires `menu.edit`.
- GET `/tenants/{tenantId}/tables` — list tables; requires `tables.view`.
- POST `/tenants/{tenantId}/tables` — create table and optionally QR asset; requires `tables.manage`.

Orders & Payments

- POST `/public/orders` — validate menu, availability, table token, totals and create pending order.
- GET `/tenants/{tenantId}/orders?status=` — list orders; requires `orders.view`.
- POST `/tenants/{tenantId}/orders/{orderId}/transition` — apply allowed status transition; requires relevant order permission.
- POST `/public/orders/{orderId}/payments` — create payment intent/session after server-side total calculation.
- POST `/webhooks/{provider}` — verify signature, deduplicate event and update payment state idempotently.
- POST `/tenants/{tenantId}/payments/{paymentId}/cash-received` — record cash receipt; requires `payments.reconcile`.

Analytics & Subscription

- GET `/tenants/{tenantId}/analytics/summary` — date-filtered metrics; requires `analytics.view`.
- GET `/tenants/{tenantId}/customers` — customer records; requires `customers.view`.
- GET `/tenants/{tenantId}/subscription` — current plan and entitlements.
- POST `/tenants/{tenantId}/subscription/checkout` — initiate plan purchase/change subject to billing rules.

Sample Order Request

- { restaurant_slug, table_token, items: [{ menu_item_id, variant_id?, addon_ids?, quantity }], customer: { name?, phone? }, payment_method }
- Client-supplied prices and totals are ignored. Server resolves current prices and returns order id, order number, verified total, order status and payment next-step if applicable.

Security Contract

- Validate permission and tenant scope on every protected request. Never rely on UI-hidden controls.
- Validate webhook signature and provider reference; reject replay/duplicate effects.
- Rate-limit public endpoints; protect QR tokens from enumeration; avoid exposing private customer data in public order status URLs.
- Return only fields needed by the caller; log sensitive administrative changes.